Dentacore Backend — NestJS Project Structure Guide

Nov 5, 2026 · @Nam

Overview

Dentacore's backend is organised by feature, with services using the ORM directly and no repository layer. This guide records where each kind of code lives and why.

Decision	Choice	Reason
Data access	No repository layer for now; services inject `PrismaService` directly	The ORM client already acts as the repository; raw SQL goes through per-feature `*.queries.ts` files
ORM	Prisma (assumed in this guide)	With TypeORM, add an `entities/` folder per feature and drop `src/prisma/`
Organisation	Feature modules, not layer folders	Each workflow's module, controller, service, DTOs and tests stay together
Integrations	Infrastructure modules (`storage/`, `messaging/`) beside `prisma/`; Stripe inside `payments/`	Thin wrappers stay free of business logic
Dependency direction	Features → infrastructure, never the reverse	Prevents circular dependencies

Why there is no Repository folder

NestJS has no data layer of its own; the ORM supplies the repository, so there is nothing to write.

- **Spring Boot:** Spring Data needs you to declare `interface PatientRepository extends JpaRepository<Patient, Long>`. One file per entity makes a `repository/` package standard.
- **TypeORM:** a generic `Repository<Patient>` is injected with `@InjectRepository(Patient)`.
- **Prisma:** `PrismaService` (extending `PrismaClient`) is the repository for every model, e.g. `this.prisma.patient.findMany()`.
- **Mongoose:** a Model injected with `@InjectModel(Patient.name)` fills the same role.

A custom repository is just another provider (e.g. `patients/patients.repository.ts`). Add one when queries become complex or repeated, when services must be ORM-free for unit testing, or under clean architecture/DDD. A repository that only passes calls through to Prisma adds files and gains nothing.

ORM deprecation risk

Breaking ORM changes are real but rare, and arrive with major versions you choose when to adopt.

- TypeORM 0.2 → 0.3 replaced `findOne(id)` with `findOneBy({ id })` and `Connection` with `DataSource`.
- Prisma 5 removed `rejectOnNotFound` in favour of `findUniqueOrThrow` / `findFirstOrThrow`.

A repository only limits the blast radius if its methods are your own abstraction (`findActivePatientsByClinic(clinicId)`), not a mirror of the ORM. Cheaper protections:

- Pin versions and read the changelog before major upgrades.
- Write integration tests for important queries against a real test database.
- Extract complex or reused queries into a repository as they appear.

Deprecation alone does not justify a repository folder at Dentacore's size.

Full project structure

The complete layout, with storage, payments, messaging, money handling and middleware included.

```
dentacore-backend/
├── prisma/
|   ├── schema.prisma                    # data models; previewFeatures =
["typedSql"]
|   ├── migrations/                      # schema changes (DDL), generated by
`prisma migrate`
|   ├── sql/                             # TypedSQL raw queries, flat; file
name = function name
|   |   ├── getPatientVisitSummary.sql
|   |   └── getDailyRevenueByDentist.sql
|   └── seed.ts                          # dev/test seed data
|
├── src/
|   ├── main.ts                          # bootstrap: rawBody, helmet,
cookie-parser, global pipes,
|   |                                    # CORS, Swagger, connectMicroservice
(RMQ), startAllMicroservices
|   ├── app.module.ts                    # root module; implements NestModule
to register middleware
|   |
|   ├── generated/
|   |   └── prisma/                      # Prisma Client output incl. sql/
typed functions (git-ignored)
|   |
|   ├── config/
|   |   ├── configuration.ts             # db, jwt, r2, stripe, rabbitmq
config
|   |   └── env.validation.ts            # fails startup if required env vars
are missing
|   |
|   ├── common/                          # cross-cutting code, no domain
knowledge
|   |   ├── decorators/                  # @CurrentUser(), @Roles(),
@Public()
|   |   ├── guards/                      # roles.guard.ts
|   |   ├── filters/                     # global exception filter (Prisma
errors → HTTP)
|   |   ├── interceptors/                # logging, response wrapping
|   |   ├── middleware/
|   |   |   └── request-id.middleware.ts # attaches x-request-id to every
request
|   |   ├── money/
|   |   |   ├── money.ts                 # Money value type: integer cents +
```

```
currency
|   |   |   └── money.spec.ts
|   |   ├── pipes/
|   |   ├── dto/                          # pagination-query.dto.ts
|   |   └── utils/
|   |
|   |   # ── Infrastructure: wrappers with no business logic ──
|   |
|   ├── prisma/
|   |   ├── prisma.module.ts              # @Global()
|   |   └── prisma.service.ts             # extends PrismaClient
|   |
|   ├── storage/                          # Cloudflare R2 (S3-compatible)
|   |   ├── storage.module.ts
|   |   ├── storage.service.ts            # upload(), getSignedUrl(), delete()
|   |   └── storage.constants.ts          # bucket names, key prefixes
|   |
|   ├── messaging/                        # RabbitMQ
|   |   ├── messaging.module.ts           # ClientsModule.registerAsync,
Transport.RMQ
|   |   └── messaging.constants.ts        # client token, queue names
|   |
|   |   # ── Features: domain data and rules ──
|   |
|   ├── auth/
|   |   ├── auth.module.ts
|   |   ├── auth.controller.ts            # /auth/login, /auth/refresh
|   |   ├── auth.service.ts
|   |   ├── auth.service.spec.ts
|   |   ├── strategies/                   # jwt.strategy.ts, local.strategy.ts
|   |   ├── guards/                       # jwt-auth.guard.ts
|   |   └── dto/                          # login.dto.ts
|   |
|   ├── users/                            # staff accounts, role assignments
|   |   ├── users.module.ts
|   |   ├── users.controller.ts
|   |   ├── users.service.ts
|   |   ├── users.service.spec.ts
|   |   └── dto/
|   |       ├── create-user.dto.ts
|   |       └── update-user.dto.ts
|   |
|   ├── patients/                         # Patient, PatientDocument, medical
```

```
history
|   |   ├── patients.module.ts           # imports StorageModule; provides
PatientsQueries
|   |   ├── patients.controller.ts       # includes document/X-ray upload
endpoints
|   |   ├── patients.service.ts          # stores object keys, not URLs; no
SQL here
|   |   ├── patients.queries.ts          # wraps raw SQL via $queryRawTyped
|   |   ├── patients.service.spec.ts
|   |   └── dto/
|   |
|   ├── appointments/                    # Appointment, AppointmentNote
|   |   ├── appointments.module.ts       # imports MessagingModule,
PaymentsModule
|   |   ├── appointments.controller.ts
|   |   ├── appointments.service.ts      # publishes appointment.created
after saving
|   |   ├── appointments.service.spec.ts
|   |   ├── events/
|   |   |   └── appointment-created.event.ts
|   |   └── dto/
|   |
|   ├── treatments/                      # TreatmentPlan, TreatmentPlanItem,
TreatmentType
|   |   ├── treatments.module.ts
|   |   ├── treatments.controller.ts
|   |   ├── treatments.service.ts
|   |   ├── treatments.service.spec.ts
|   |   └── dto/
|   |
|   ├── payments/                        # Payment, Invoice, InvoiceLineItem
(Stripe)
|   |   ├── payments.module.ts           # exports PaymentsService only
|   |   ├── payments.controller.ts       # POST /payments/checkout, GET
/payments/:id
|   |   ├── payments.service.ts          # payment records, links to
appointments
|   |   ├── payments.queries.ts          # raw SQL reports (e.g. daily
revenue by dentist)
|   |   ├── payments.service.spec.ts
|   |   ├── stripe.service.ts            # SDK wrapper, private to this
module
|   |   ├── webhooks.controller.ts       # POST /payments/webhook (@Public,
```

```
raw body)
|   |   ├── pricing/
|   |   |   ├── deposit.calculator.ts    # calculateDeposit(plan): Money
|   |   |   ├── invoice-totals.ts        # subtotal, discounts, total
|   |   |   └── *.spec.ts
|   |   ├── events/
|   |   |   └── payment-succeeded.event.ts
|   |   └── dto/
|   |
|   └── notifications/                   # RabbitMQ consumer
|       ├── notifications.module.ts      # registers the consumer under
controllers
|       ├── notifications.consumer.ts    # @Controller() with @EventPattern
handlers
|       ├── notifications.service.ts     # email/SMS sending
|       └── notifications.service.spec.ts
|
├── test/
|   ├── app.e2e-spec.ts
|   ├── payments.e2e-spec.ts             # webhook signature + idempotency
|   └── jest-e2e.json
|
├── docker-compose.yml                   # local Postgres + RabbitMQ (also
needed for `prisma generate --sql`)
├── .env                                 # git-ignored
├── .env.example                         # DATABASE_URL, JWT_*, R2_*,
STRIPE_*, RABBITMQ_URL
├── .eslintrc.js
├── .prettierrc
├── nest-cli.json
├── package.json
├── tsconfig.json
└── tsconfig.build.json
```

Unit tests (`*.spec.ts`) sit beside the code they test; only end-to-end tests live in `test/` . `nest g resource <name>` scaffolds most of a feature folder.

Modules: per feature, not per entity

Make one module per feature (a workflow with its own endpoints and rules); several entities can share it.

Give an entity its own module when clients call its endpoints directly (`/patients`), it has its own lifecycle (booked → confirmed → completed or cancelled), or other features use it through a service.

Keep it in its parent's module when it only makes sense through the parent and its routes nest under it (`/appointments/:id/notes`).

Module	Entities inside
`patients/`	Patient, PatientDocument, medical history
`appointments/`	Appointment, AppointmentNote
`treatments/`	TreatmentPlan, TreatmentPlanItem, TreatmentType
`payments/`	Payment, Invoice, InvoiceLineItem
`users/`	User (staff), role assignments

- Not every module needs all three files: `notifications/` has a queue consumer and no HTTP controller; `prisma/` , `storage/` and `messaging/` have only a module and a service.
- A large feature splits inside its folder (e.g. `appointments.service.ts` + `availability.service.ts`), not into more modules.
- Needing `forwardRef()` to break a circular dependency usually means one workflow was split across two modules.

Infrastructure vs feature modules

- **Infrastructure** wraps a third-party API or connection and holds no business logic: `prisma/` , `storage/` , `messaging/` . Name by role, not provider (`storage/` , not `r2/`).
- **Feature** owns domain data and rules: `patients/` , `appointments/` , `payments/` .
- Features import infrastructure and each other's exported services. Infrastructure never imports features.

Integrations

R2 and RabbitMQ get infrastructure modules; Stripe lives inside the `payments/` feature because payments own domain data.

Storage (Cloudflare R2)

- `StorageService` uses `@aws-sdk/client-s3`, since R2 is S3-compatible, configured with R2's endpoint and credentials.
- Feature services call `storageService.upload(...)` and never learn the provider.
- Store only the object key in the database (e.g. `patients/123/xray-2026-11.png`) and generate signed URLs on request. Medical images must not sit in a public bucket.

Payments (Stripe)

- `payments/` is a full feature: it stores payment records, links them to appointments or invoices, and reacts to Stripe events.
- `stripe.service.ts` is private to the module; other modules import `PaymentsModule` and call `PaymentsService`.
- Webhook signature checks need the raw body: create the app with `NestFactory.create(AppModule, { rawBody: true })` and pass `req.rawBody` to `stripe.webhooks.constructEvent(...)`.
- The webhook route skips the JWT guard via `@Public()`.
- Store each Stripe event ID and skip duplicates, because Stripe retries deliveries.

Messaging (RabbitMQ)

Uses Nest's built-in RMQ transport from `@nestjs/microservices`.

- `messaging/` registers the client with `ClientsModule.registerAsync` and `Transport.RMQ` and exports it.
- Event payloads belong to the producer (`appointments/events/appointment-created.event.ts`); consumers import that type.
- Consumers live in the reacting feature. `@EventPattern` handlers only register inside a `@Controller()` class listed in the module's `controllers`.
- `main.ts` runs a hybrid app: `app.connectMicroservice({ transport: Transport.RMQ, options: { urls, queue } })`, then `await app.startAllMicroservices()` before `app.listen()`.

Caveats:

- The built-in transport uses one queue per connection with pattern routing; it does not control exchanges, routing keys or fan-out to several queues.

- Publish only after the database write succeeds. For payment events, use the outbox pattern: write the event in the same transaction and publish from that table.
- Make consumers idempotent, because RabbitMQ can redeliver.
- Heavy consumers can later move to a separate worker process; only `main.ts` changes.

Domain logic: money calculations

Generic money handling goes in `common/money/`; business rules about money go in the feature that owns them.

Kind	Examples	Location
Generic money handling	`Money` type, add/subtract, rounding, formatting, conversion to cents	`common/money/`
Business rules	20% deposit on a treatment plan, returning-patient discount, instalments	`payments/pricing/`

- Other features get pricing results through `PaymentsService`, never by importing `payments/pricing/` directly.
- Write calculators as plain functions or classes, not `@Injectable()` services, so unit tests need no Nest testing module. Make them injectable only when they need dependencies such as `ConfigService`.
- Store and calculate in integer cents (Prisma `Int`), never floats; Stripe also expects the smallest currency unit.
- Round once at a defined point (e.g. per line item) and test that rule, so invoice totals match Stripe charges.
- Nest's `libs/` folder (`nest g library`) is for sharing code between several deployable apps. With one app, plain folders in `src/` are enough.

Raw SQL queries

Raw SQL files live in `prisma/sql/` and are called from a `*.queries.ts` provider in the feature that owns them, using Prisma's TypedSQL.

TypedSQL workflow: write a query in `prisma/sql/`, run `npx prisma generate --sql` against a live database, then call the generated function with `$queryRawTyped` for typed results (Prisma).

```
-- prisma/sql/getPatientVisitSummary.sql
-- @param {String} $1:clinicId
SELECT p.id, p.full_name, COUNT(a.id)::int AS visit_count
FROM patients p
LEFT JOIN appointments a ON a.patient_id = p.id
WHERE p.clinic_id = $1
GROUP BY p.id;
```

```
// src/patients/patients.queries.ts
@Injectable()
export class PatientsQueries {
  constructor(private prisma: PrismaService) {}

  visitSummary(clinicId: string) {
    return this.prisma.$queryRawTyped(getPatientVisitSummary(clinicId));
  }
}
```

- `getPatientVisitSummary` is imported from the generator's output (e.g. `../generated/prisma/sql`).
- A `*.queries.ts` file is the one place a repository-style layer earns its keep: SQL stays out of services, and tests can mock `PatientsQueries` without a database. Register it as a provider in the feature module, and only in features that use raw SQL.
- Ordinary Prisma Client calls still go straight through `PrismaService`.
- TypedSQL is still a Preview feature in Prisma ORM 7 (`previewFeatures = ["typedSql"]`); pin the Prisma version.
- `prisma generate --sql` needs a running database, so CI must start one before generating (locally, `docker-compose.yml`).
- Never concatenate user input into SQL or pass it to `$queryRawUnsafe`; TypedSQL's `$1` parameters are safe.
- Schema changes (DDL) are migrations, not queries: run `prisma migrate dev --create-only`, then edit `migration.sql`.

Middleware

Dentacore needs little middleware: `helmet` and `cookie-parser` in `main.ts`, plus one `request-id.middleware.ts`.

- **Third-party** (`helmet`, `cookie-parser`, `compression`): `app.use()` in `main.ts`.
- **Custom cross-cutting:** a class in `common/middleware/`, registered in `AppModule.configure()` with `consumer.apply(RequestIdMiddleware).forRoutes('*')`.
- **Feature-specific:** in that feature's folder, registered in its module with `forRoutes(PatientsController)`. Rare in practice.

Nest's request lifecycle decides what goes where:

1. Middleware
2. Guards
3. Interceptors (before)
4. Pipes
5. Route handler
6. Interceptors (after)
7. Exception filters

Middleware runs before Nest picks a handler, so it cannot read decorators like `@Roles()` or `@Public()` or see the handler's result.

Job	Use	Location
JWT checks, roles, `@Public()` bypass	Guard	`auth/guards/`, `common/guards/`
Request timing logs, response wrapping	Interceptor	`common/interceptors/`
DTO validation and transformation	Pipe	Global `ValidationPipe` in `main.ts`
Errors → HTTP responses	Exception filter	`common/filters/`
Request IDs, security headers, cookie parsing	Middleware	`main.ts`, `common/middleware/`

Pass the request ID into logs and RabbitMQ messages to trace a booking from HTTP request to notification. The Stripe webhook needs no middleware because `rawBody: true` already provides the raw body. Auth checks written in middleware belong in a guard.